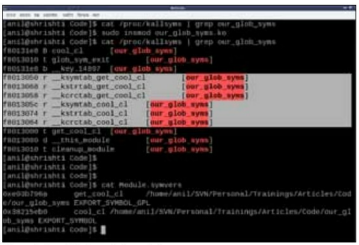

Figure 1: Our global symbols module

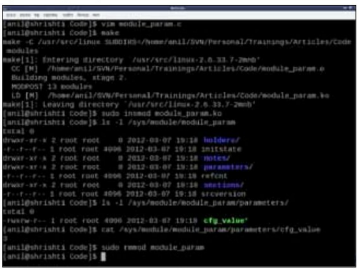

Figure 2: Experiments with the module parameter

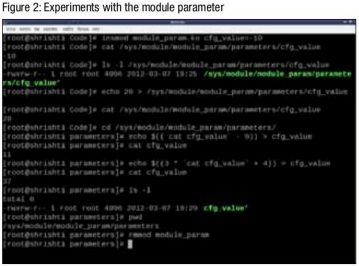

Figure 3: Experiments with the module parameter (as root)

(*__ksymtab*), kernel string table (*__kstrtab*), and kernel CRC table (*__kcrctab*) sections, marking it to be globally accessible. Figure 1 shows a filtered snippet of the */proc/kallsyms* kernel window, before and after loading the module *our_glob_syms.ko*, which has been compiled using the usual driver *Makefile*.

The following code shows the supporting header file (*our_glob_syms.h*), to be included by modules using the exported symbols *cool_cl* and *get_cool_cl*:

```
#ifndef OUR_GLOB_SYMS_H
#define OUR_GLOB_SYMS_H

#ifdef __KERNEL__
#include <linux/device.h>

extern struct class *cool_cl;
extern struct class *get_cool_cl(void);
#endif

#endif
```

Figure 1 also shows the file *Module.symvers*, generated by compiling the module *our_glob_syms*. This contains the various details of all the exported symbols in its directory. Apart from including the above header file, modules using the exported symbols should possibly have this file *Module.symvers* in their build directory.

Note that the *<linux/device.h>* header in the above examples is being included for the various class-related declarations and definitions, which have already been covered in the earlier discussion on character drivers.

Module parameters

Being aware of passing command-line arguments to an application, it would be natural to ask if something similar can be done with a module—and the answer is, yes, it can. Parameters can be passed to a module while loading it, for instance, when using *insmod*. Interestingly enough, and in contrast to the command-line arguments to an application, these can be modified even later, through sysfs interactions.

The module parameters are set up using the following macro (defined in *<linux/moduleparam.h>*, included through *<linux/module.h>*):

```
module_param(name, type, perm)
```

Here, *name* is the parameter name, *type* is the type of the parameter, and *perm* refers to the permissions of the *sysfs* file corresponding to this parameter. The supported type values are: *byte*, *short*, *ushort*, *int*, *uint*, *long*, *ulong*, *charp* (character pointer), *bool or invbool* (inverted Boolean). The following module code (*module_param.c*) demonstrates a module parameter:

```
#include <linux/module.h>
#include <linux/kernel.h>

static int cfg_value = 3;
module_param(cfg_value, int, 0764);

static int __init mod_par_init(void)
{
    printk(KERN_INFO "Loaded with %d\n", cfg_value);
    return 0;
```